A Implementation Details

A.1 SICO

A.1.1 Feature extraction

In feature extraction step, we instruct LLM to extract 5 features and calculate the utility score $\mathcal{U}$ of prompts encompassing each of these features. Then we select the feature with the highest utility for further steps. The goal of this step is to find a good feature to accelerate process, and make the whole process stable. Because sometimes LLM cannot extract useful features to evade detectors. The pseudo-code illustrating this selection process is outlined in Algorithm 3. Table 8 presents the prompt template used for feature extraction. Here, K text pairs generated by AI and Human are positioned within their respective positions. Table 21 shows the examples for feature extracted by LLM.

Algorithm 3 Feature selections

Require: list of features T_{feature}, prompt utility function $\mathcal{U}(\cdot)$
Initialize $t^*_{\text{feature}} \leftarrow \emptyset$
Initialize $\mathcal{U}_{max} \leftarrow -\infty$
for each feature $t_{\text{feature},i}$ in T_{feature} **do**
  Construct prompt $p_i \leftarrow t_{\text{feature},i} \oplus p_{\text{task}}$
  if $\mathcal{U}(p_i) > \mathcal{U}_{max}$ **then**
    $t^*_{\text{feature}} \leftarrow t_{\text{feature},i}$
    $\mathcal{U}_{max} \leftarrow \mathcal{U}(p_i)$
  end if
end for
return t^*_{feature}

Table 8: Prompt for feature extraction.

Here are the writings from AI and human: AI writing: <AI-generated text> Human writing: <human-written text> ... What is the key distinct feature of human's writings? [LLM complete]

LLM consistently extract useful features. To test if LLM can reliably extract useful features, we conducted three separated experiments by running three feature extractions on different sets of human-written text and AI-generated text. We use three different extracted features to guide LLM generation and test the AUC drop after adopting 3 different features compared with the originally generated text on the Writing task. Table 9 shows the results, indicating that LLM consistently extracts useful features for detector evasion from different examples.

Table 9: AUC drop of different features extracted based on different human-written and AI-generated text.

Orig.	Feature 1	Feature 2	Feature 3
0.908	-0.288	-0.261	-0.142

A.1.2 Task instructions

Table 10 shows the actual task instruction p_{task} we used in SICO. As mentioned in Section 3, feature text t_{feature} at first step will be inserted before these task instructions. The "Paraphrase" instruction is p_{para} used in paraphrase generation for substitution (Line 6 of Algorithm 1).

A.1.3 Word Substitution

We employ WordNet synsets to derive synonyms for given words. During optimization of in-context examples, we only substitute content words, namely nouns, verbs, adjectives, and adverbs. Furthermore, we part-of-speech (POS) tag of the synonym to ensure it aligns with the original word. For POS tagging, we utilize the Stanford POS Tagger (Toutanova et al., 2003). Additionally, to maintain fluency in the modified text after substitution, we employed a pretrained mask language model to exclude synonyms with low likelihood. In experiment we use RoBERTa-base model (Liu et al., 2019).

Table 10: Task instructions of each task.

Task	**Task instruction** p_{task}
Writing	Based on the description, complete an academic paragraph in human style writings: Prompt: < *task input* > Human: [LLM complete]
QA	Based on the description, answer questions in human style writings: Q: < *task input* > Human: [LLM complete]
Review	Based on the description, write a human review about the given object and keywords, with a specified sentiment: Object, Keywords, Sentiment: < *task input* > Human: [LLM complete]
Paraphrase	Based on the description, rewrite this in human-style writings: Origin: < *original AI-generated text* > Human: [LLM complete]

A.2 Baselines

A.2.1 DIPPER

We choose the best evasion performance parameter setting from the original paper (Krishna et al., 2023), which is 60 for lexical diversity and 60 for re-ordering. And we set sampling temperature to 0.75, following the original implementation.

A.2.2 Human prompt

We carefully design a paraphrase prompt based on the detection idea of GPTzero (Tian, 2023) and prompt shared online (Uploader, 2023), which distinguish the AI-generated content from Human-written by *perplexity* and *burstiness*, stated by its creator[4]. *Perplexity* is the concept raised in NLP field, which measures how well a language model predicts a text sample. A text with a lower perplexity score indicates that the language model is better at calculating the next word that is likely to occur in a given sequence. On the other hand, *burtiness* basically measures the variation between sentences, including sentence length and structures. The lower the values for these two factors, the more likely it is that a text was produced by an AI. Table 11 shows the prompt we designed.

[4]https://theconversation.com/we-pitted-chatgpt-against-tools-for-detecting-ai-written-text-and-the-results-are-troubling-199774

Table 11: Human-designed prompt to evade AI-generated text detection.

When it comes to writing content, two factors are crucial, "perplexity" and "burstiness". Perplexity measures the complexity of text. Separately, burstiness compares the variations of sentences. Humans tend to write with greater burstiness, for example, with some longer or complex sentences alongside shorter ones. AI sentences tend to be more uniform. Paraphrase the following AI sentence to be human-like, with a good amount of perplexity and burstiness: Orig: < *orignal AI-generated text* > New: [LLM complete]

B Extra Experiments

B.1 Semantic preserving

We measure semantic similarity using t5-based sentence encoder (Ni et al., 2022), which leads in semantic text similarity task of MTEB benchmark (Muennighoff et al., 2023). Table 12 reports a comparison of the cosine similarity of text before and after paraphrasing by different methods. Our methods successfully preserves the

Table 12: Cosine similarity between original AI-generated text and their respective paraphrased versions using different methods. The best scores in each task are highlighted in **bold**.

Method	Writing	QA	Review
Parrot	0.964	0.961	0.966
DIPPER	0.956	0.941	0.940
GPT-Para	**0.986**	**0.982**	**0.987**
Human Prompt	0.979	0.968	0.978
SICO-Para	0.976	0.964	0.971

semantic meaning during paraphrasing, and beats the other specifically trained paraphraser. Paraphrasing directly using GPT-3.5 yields the most promising results.

B.2 Stability of SICO

SICO is able to consistently construct effective detection evasion prompts, irrespective of the diversity in the initial AI-human text pairs and randomized samples drawn from the LLMs. This demonstrates the effectiveness of SICO in various initial conditions and settings, highlighting its applicability to diverse scenarios. Figure 3 presents the detection evasion performance of the best prompt at each step, denoted as $\mathcal{U}(p^*)$ in Equation 1, where a higher value means higher evasion performance. The prompt is evaluated by 50 validation examples. We run three distinct experiment for each task to draw the plot. SICO successfully optimizes the initial prompt (at step 0) and achieves a high level of evasion performance across all three tasks, with different in-context examples.

B.3 SICO performs better against more capable detectors

We use the detectors' performance on the original AI-generated text to represent their base capability. The SICO advancing performance is measured by the AUC difference between best of SICO-Para or SICO-Gen and the best-performing paraphraser baselines. The statistical Pearson correlation is 0.47 with a p-value of 0.048, indicating a moderate positive correlation.

B.4 SICO with different LLMs

To examine the effectiveness of SICO employing different LLMs, we adopted an additional experiment which employs SICO-Para on three different LLMs: WizardLM-13B Xu et al. (2023), GPT-3.5-turbo-0301, and